Lecture 4:
Baby's First Driver

Documentation methods, file structures, and driver theory.

Robojackets Firmware Training 2026

Administrivia - Training Outline

- Lecture 1: C/C++ Basics
- Lecture 2: Interrupts & Analog I/O
- Lecture 3: Communication Protocols
- **Lecture 4: Driver Development (today)**
- Lecture 5: State Machines & RTOS
- Practical Soldering Training

Drivers

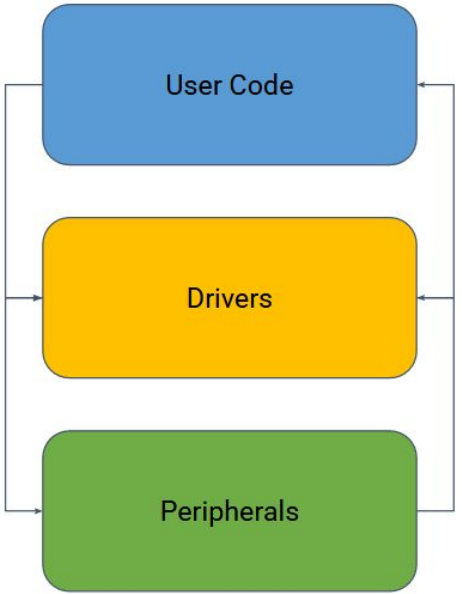
What is a Driver?

A driver is a layer of abstraction between a physical device and the software the is using it.

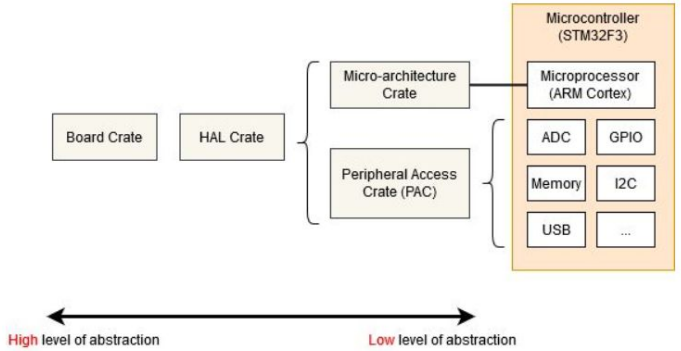
A driver provides:

- Functions to initialize and control the device
- Relevant data and data structures to use the device
- Basic information on how the device works

The goal is to make using the device as simple as possible for the next layer up.



Abstraction Layers

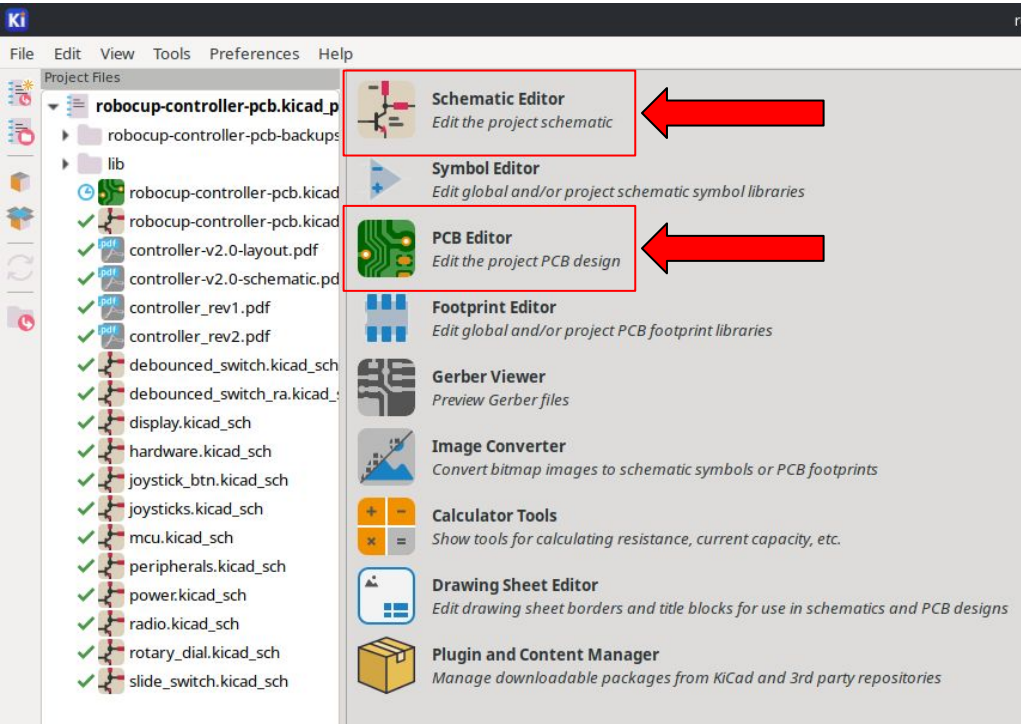


- Silicon - Bare metal
- Hardware Abstraction Layer (HAL)
  - Memory access simplification
  - Peripheral access simplification
  - Often provided
  - (uin32_t*) 0x4002_2300 |= 0xF -> digitalWrite(14, HIGH)
- Drivers
  - Sometimes use the HAL, sometimes don't
  - Wrap primitive peripherals into actual devices
  - Sometimes findable
  - I2C port -> Screen Controller
- User Level Program
  - Utilizes functions from all layers below to create a functional net of devices

The Lowest Level

KiCAD

KiCAD is the PCB design software used by most teams. In an ideal world you would not need to touch this, but let's be real.

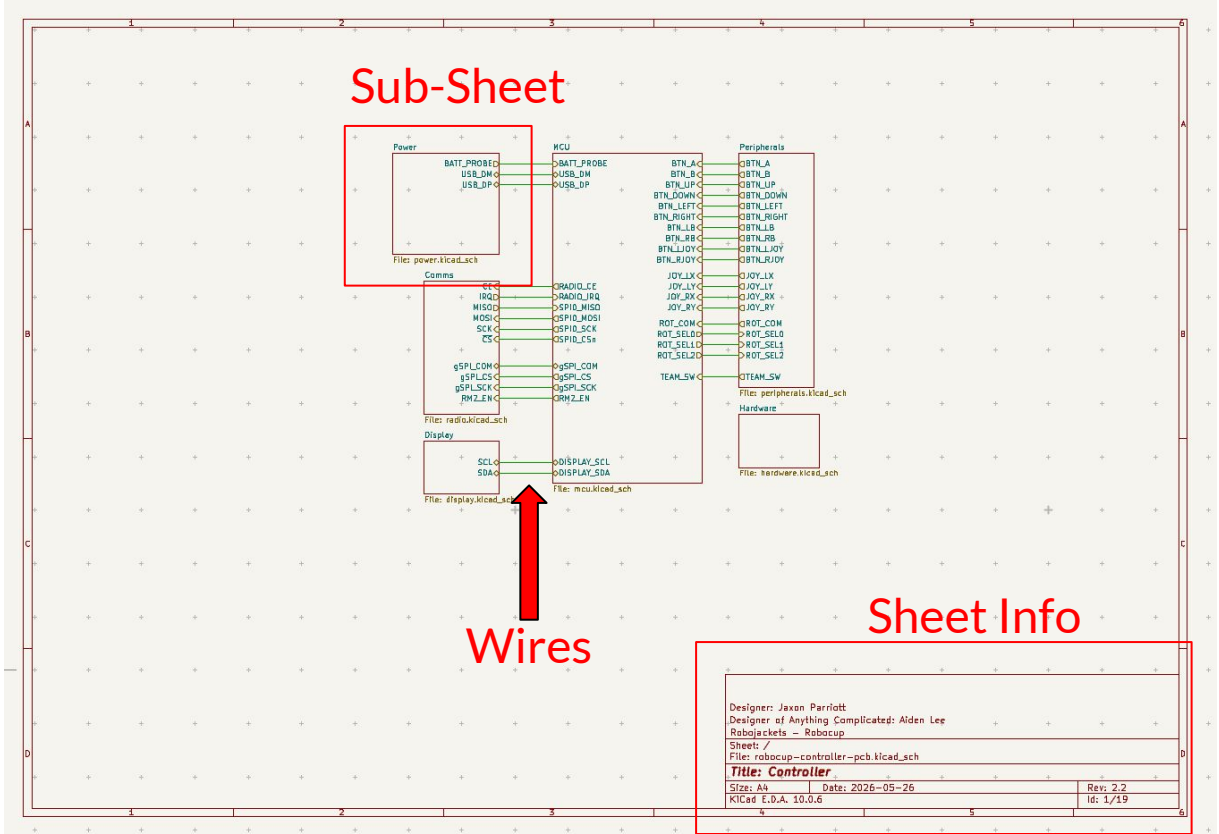


- Schematic Editor
  - Simplest block diagram of the PCB
  - Useful for checking:
    - Connections
    - Component Info
- PCB Editor
  - Real layout and routing of the PCB
  - Useful for checking:
    - Physical Layout

Install: https://www.kicad.org/download/

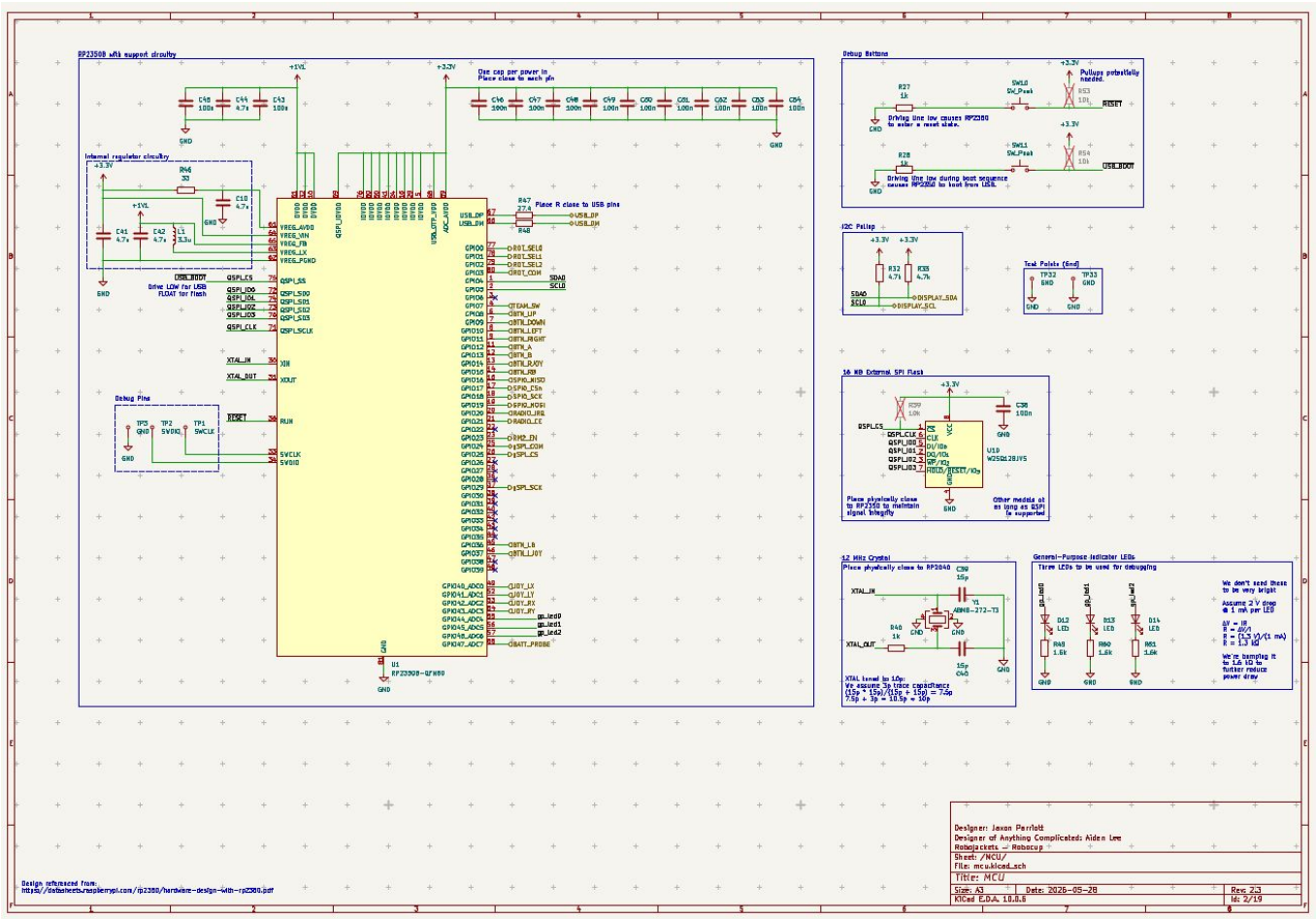
Schematic Editor I

The first sheet that opens should be the top-level sheet. While not always, it will likely lead to many sub-sheets.

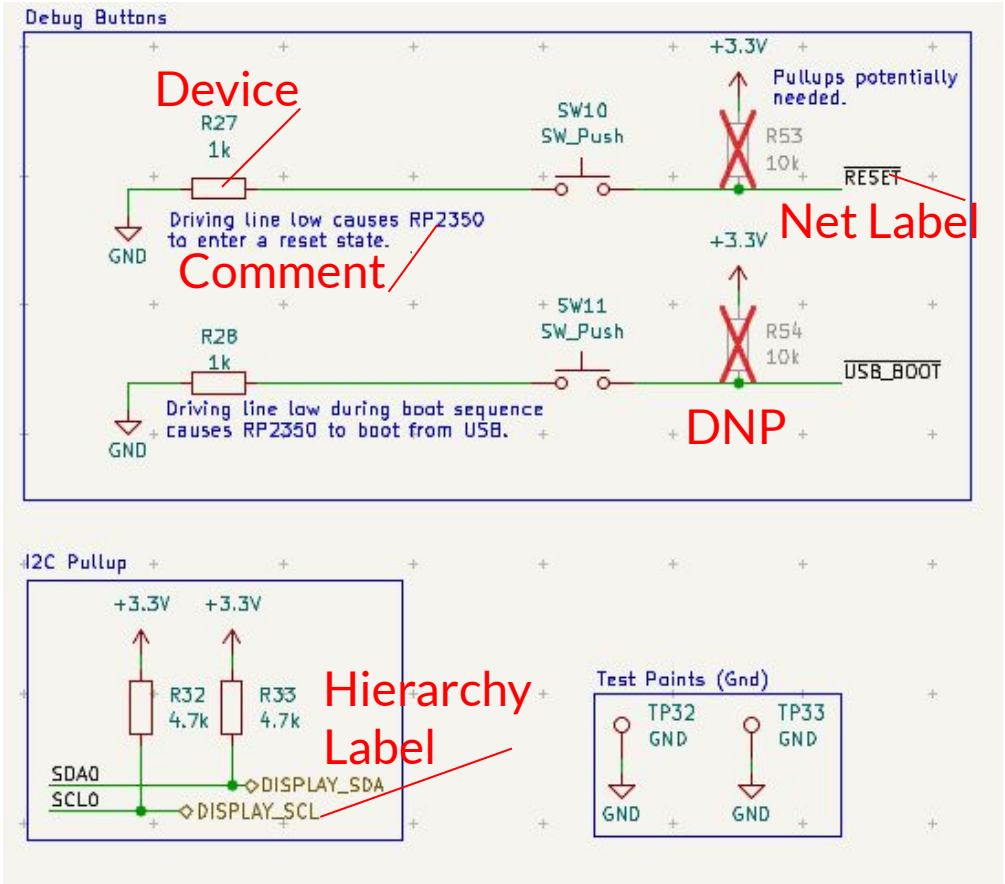


- Sub-sheet
  - A link to a second sheet with defined inputs/outputs
  - Used to abstract
- Sheet Info
  - Outlines important info about the current sheet.
- Wires
  - Connects objects

Schematic Editor II



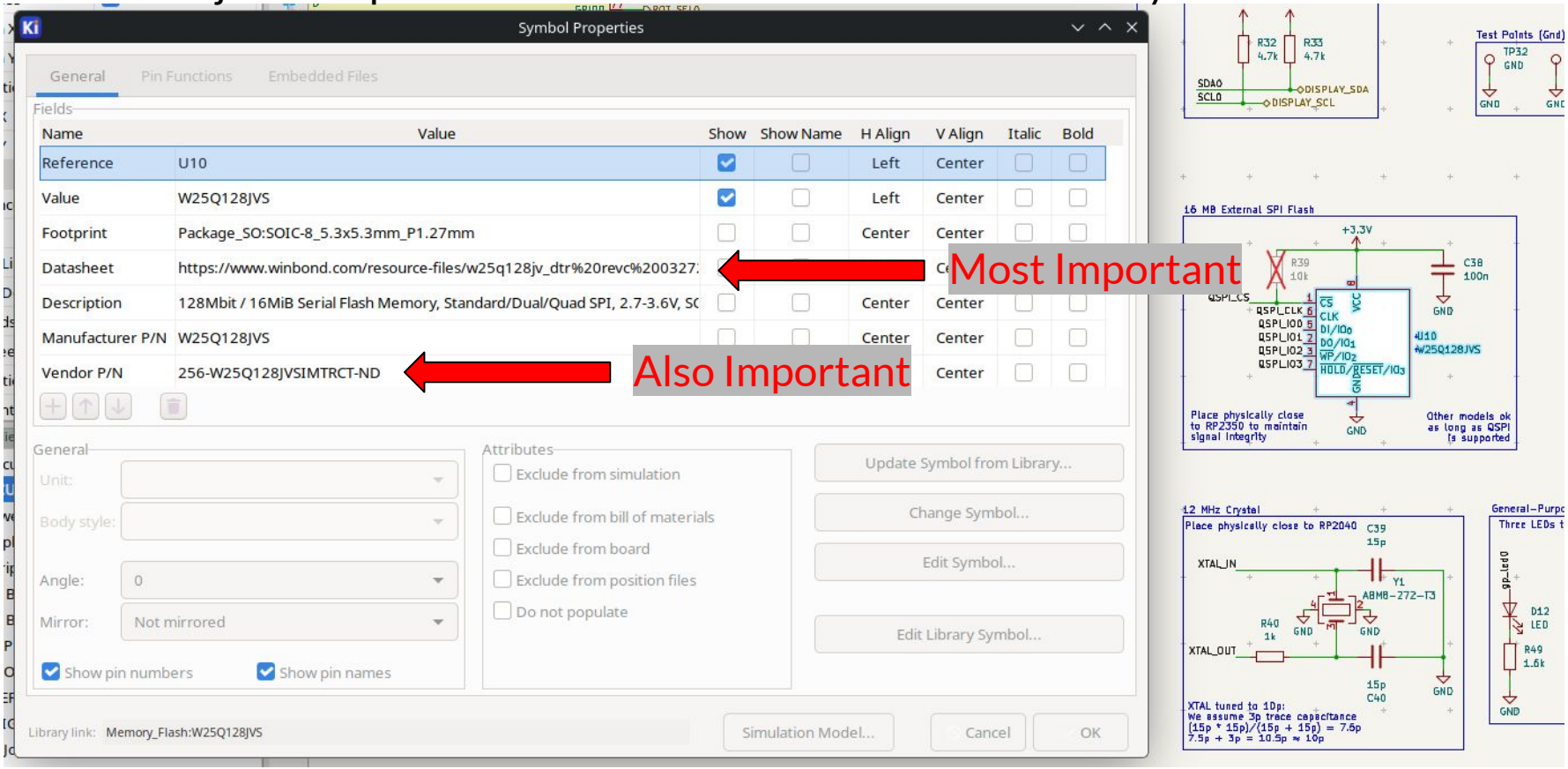
Schematic Editor II.1



- Object
  - Unique ID
  - Value
- Comments
  - Blue
  - Pray they are good
- Net Labels
  - Function as wires without having to route them.
- Hierarchy Labels
  - Link internal sub-sheet wires to external ports
  - Input/Output/Bidirectional/Tristate/Passive
- Do Not Populate
  - Marks device to be given pads on the pcb, but not actually soldered on.

Schematic Editor II.2

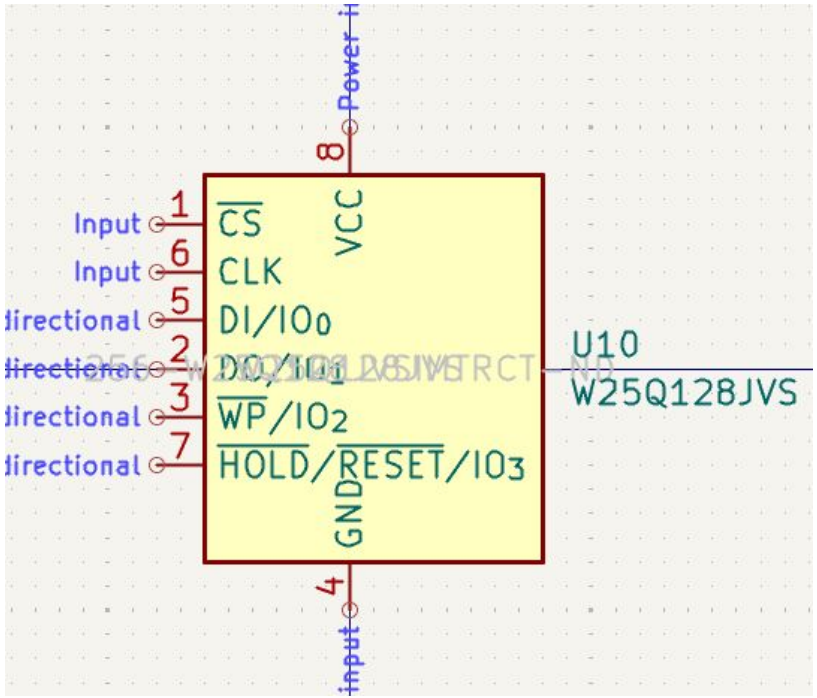
Select a object and press “e” to see more information on it. Pray it is filled out.



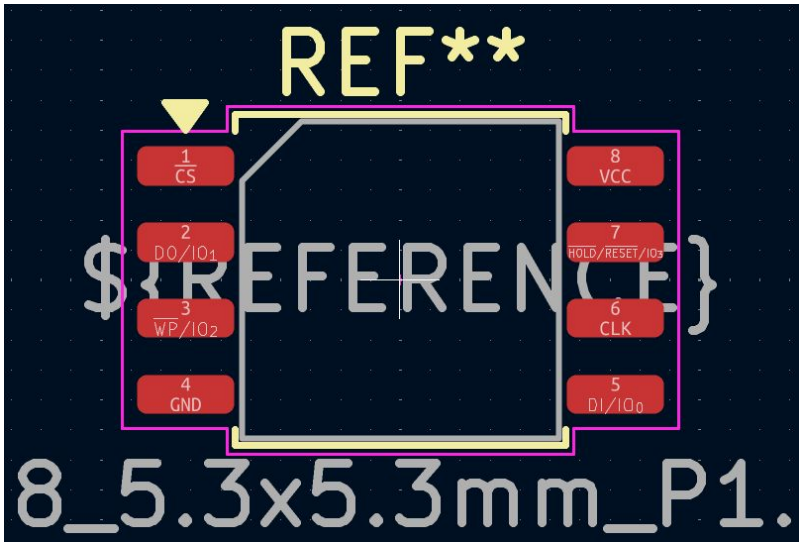
Matching Pins to Physical Pins

Always double check pin connections as pin names are not guaranteed to match, only pin numbers!

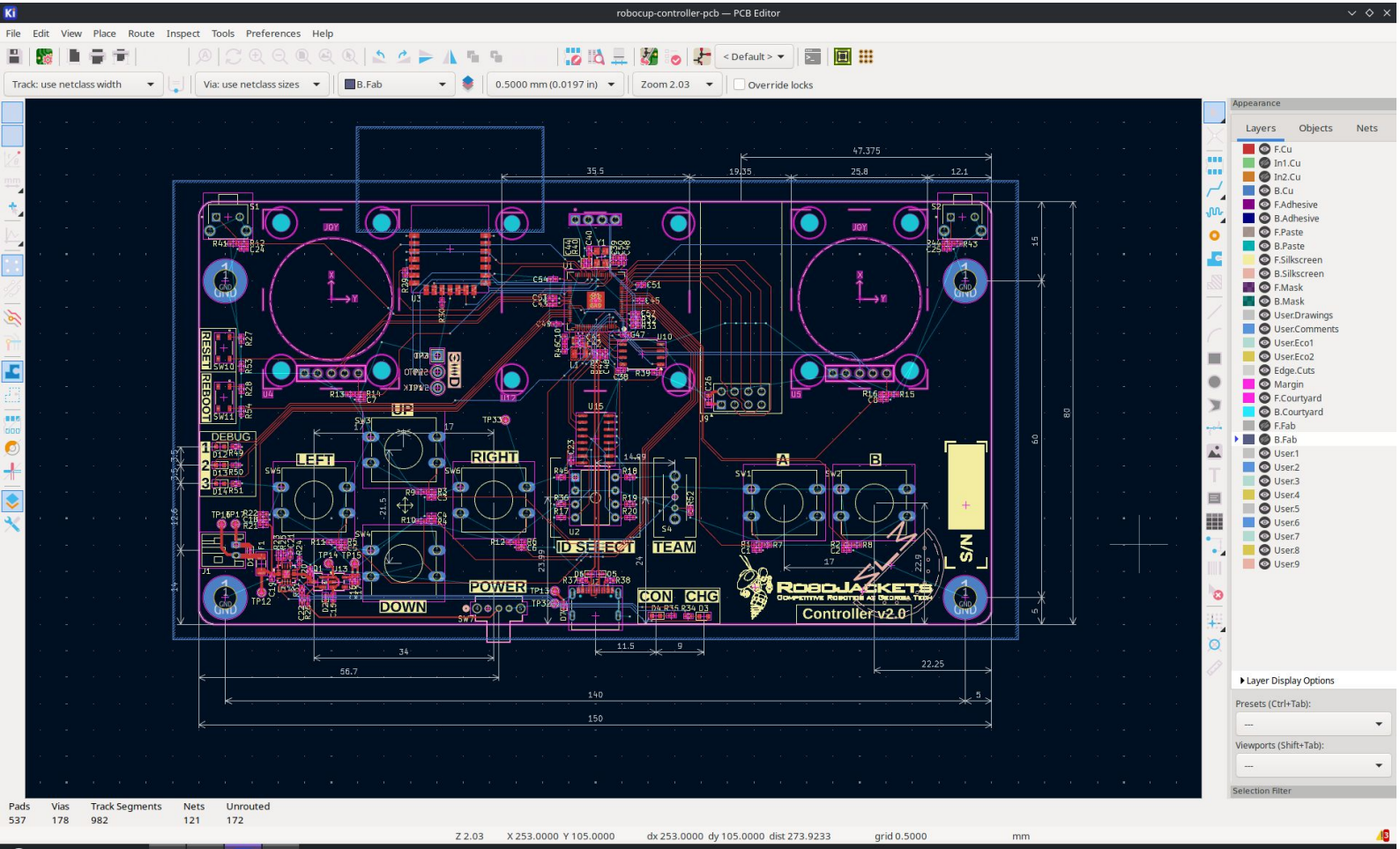
Symbol



Footprint

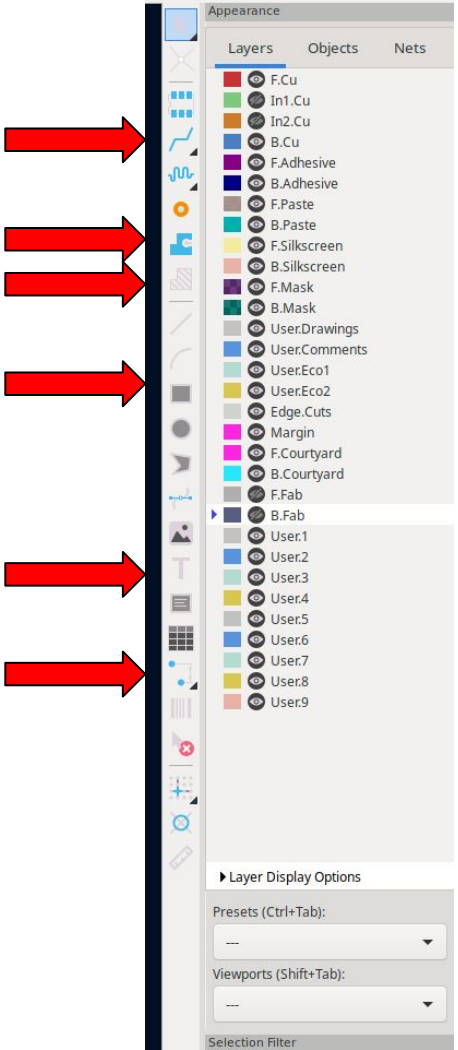


PCB Editor


robocup-controller-pcb — PCB Editor
File Edit View Place Route Inspect Tools Preferences Help
Track: use netclass width
Via: use netclass sizes
B.Fab
0.5000 mm (0.0197 in)
Zoom 2.03
Override locks
Appearance
Layers Objects Nets
F.Cu
In1.Cu
In2.Cu
B.Cu
F.Adhesive
B.Adhesive
F.Paste
B.Paste
F.Silkscreen
B.Silkscreen
F.Mask
B.Mask
User.Drawings
User.Comments
User.Eco1
User.Eco2
Edge.Cuts
Margin
F.Courtyard
B.Courtyard
F.Fab
B.Fab
User.1
User.2
User.3
User.4
User.5
User.6
User.7
User.8
User.9
Layer Display Options
Presets (Ctrl+Tab):
Viewports (Shift+Tab):
Selection Filter
UP
DOWN
LEFT
RIGHT
A
B
POWER
ID SELECT
TEAM
CON CHG
RESET
REBOOT
DEBUG
S/N
RoboJackets
Competitive Robotics At Georgia Tech
Controller v2.0
47.375
35.5
19.35
25.8
12.1
80
60
56.7
34
11.5
9
22.25
140
150
Pads 537
Vias 178
Track Segments 982
Nets 121
Unrouted 172
Z 2.03
X 253.0000 Y 105.0000
dx 253.0000 dy 105.0000 dist 273.9233
grid 0.5000
mm

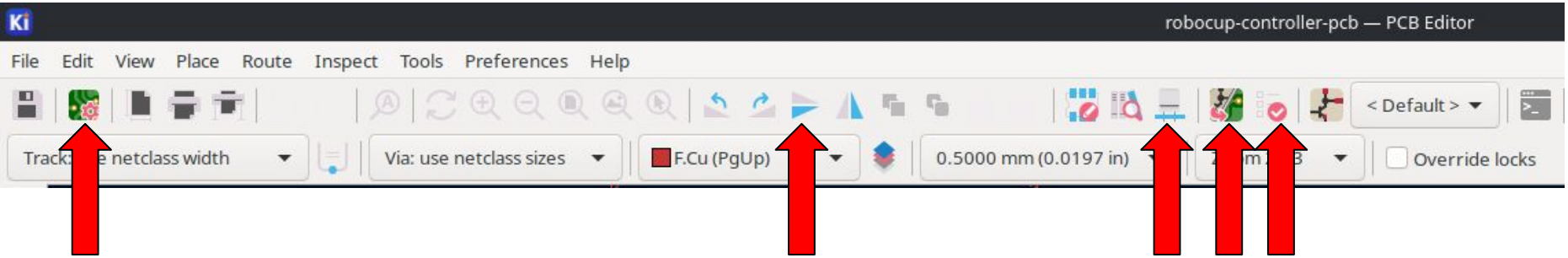
PCB Editor I

- Route wire
  - “X”
- Draw Fill
  - Fill areas are “filled” with a net.
- Draw Rule Zone
  - Used to mark exclusion zones
- Draw Shapes
- Draw Text
- Draw Ruler
  - Marks distance between points.



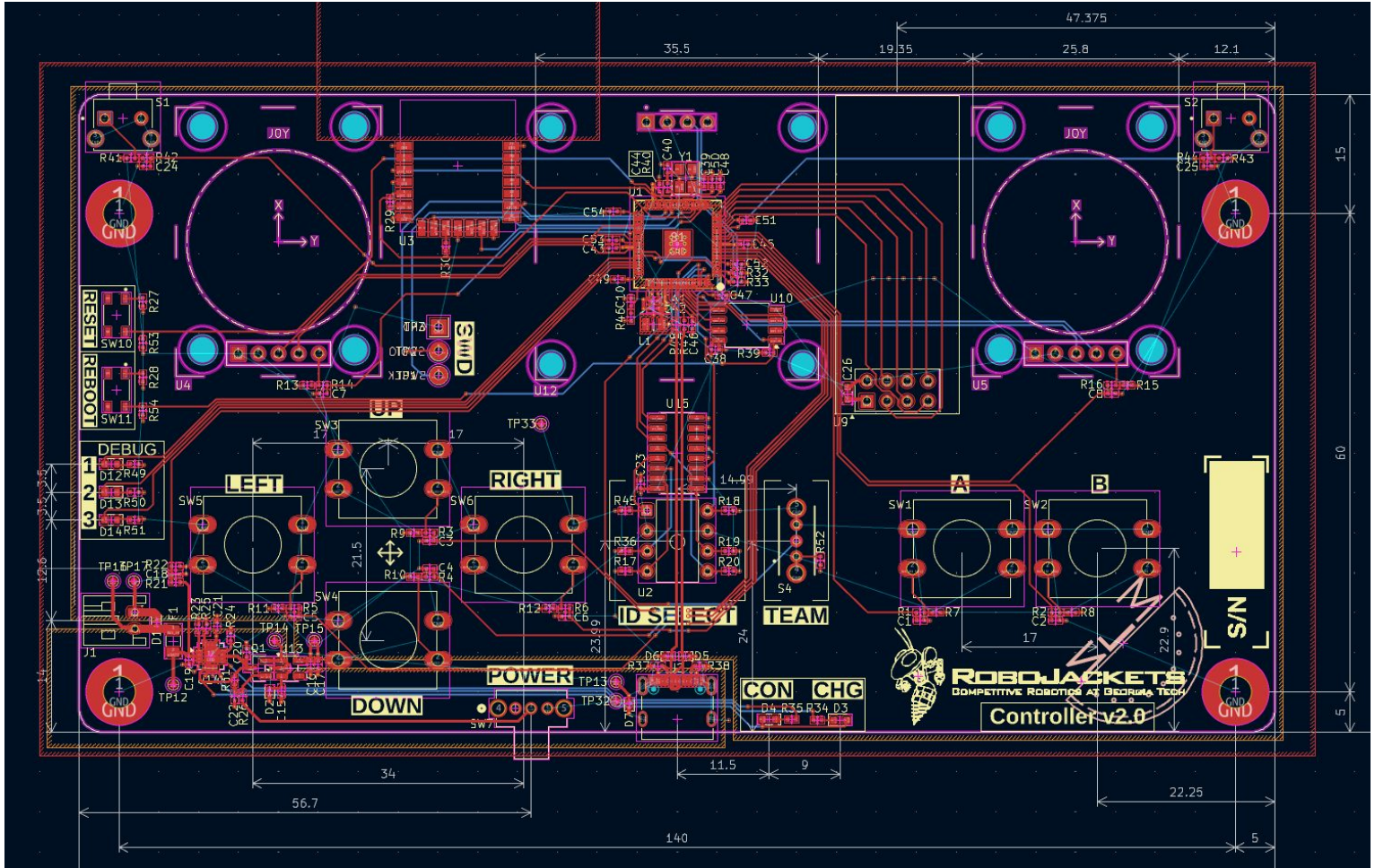
- Appearance
  - Select what layers/objects/nets are visible.
  - Select which layer you are currently editing.

PCB Editor II



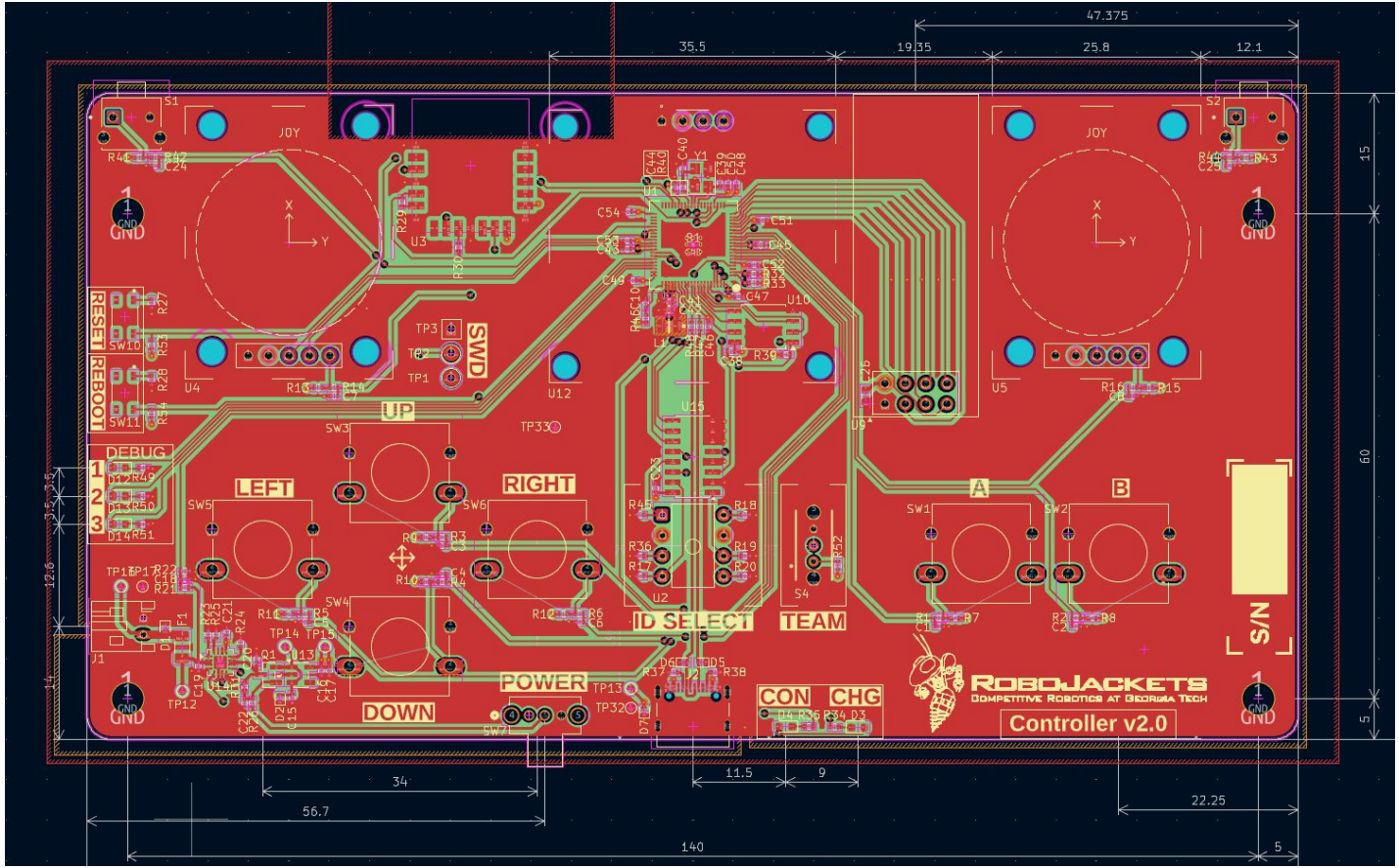
- Board Settings
  - Sets various physical characteristics and design rules for the board
  - Used to setup the stackup
- Rotations/Mirrors
  - Also have various hotkeys
  - Note the lack of “flip”
- 3D-Viewer
- Update from Schematic
  - Has to be run anytime something is changed in the schematic
- Run DRC
  - Runs all design rule checks

PCB Editor III.1



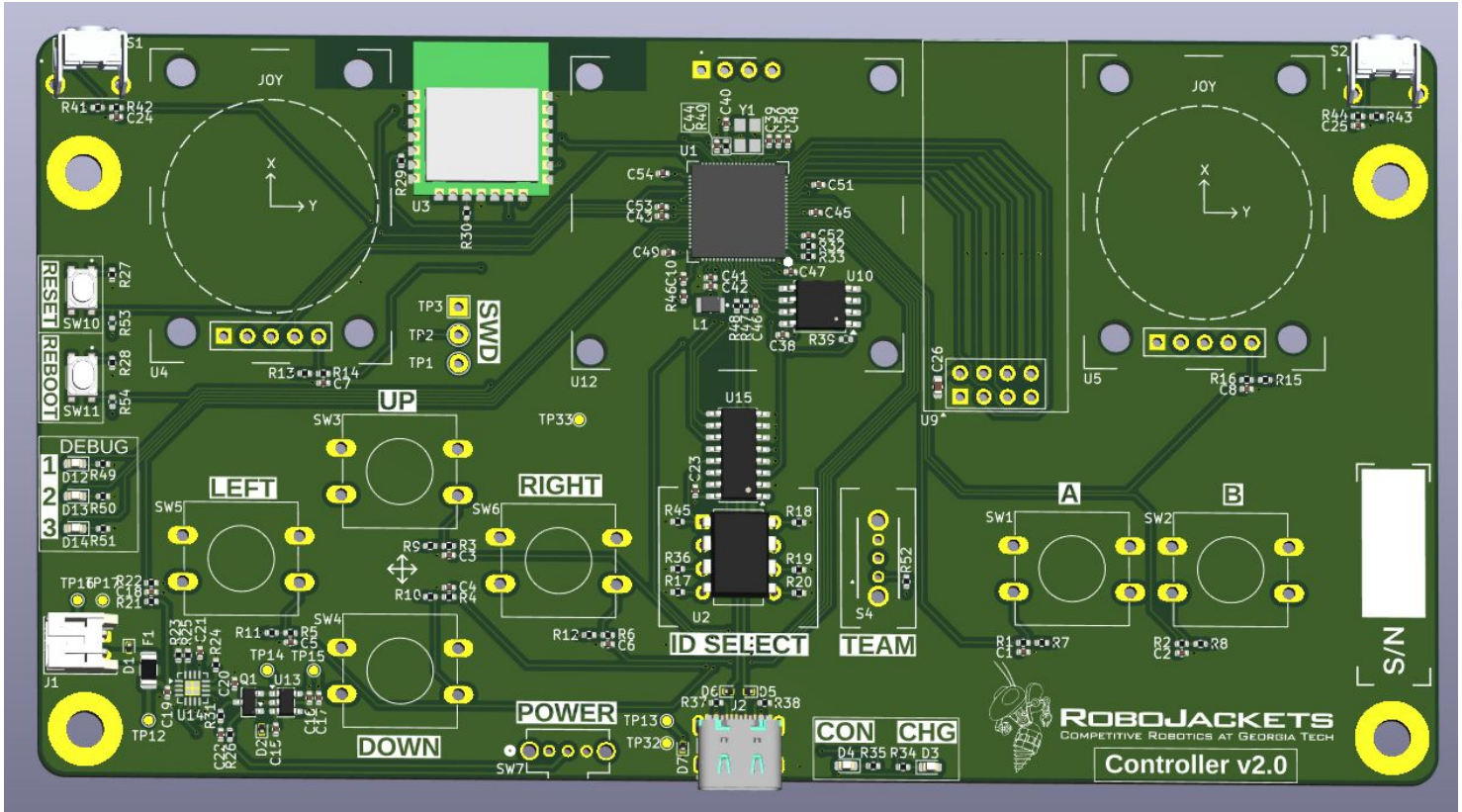
PCB Editor III.2

Fill zones can be filled with “b” and unfilled with “ctrl+b”



3D Viewer

Lets you see a real view of what the board will look like. Good to check silkscreen and layout. You can highlight devices here and they will highlight in pcb/schem.



Reading 101

Technical Documents

- Datasheets
  - A high level overview of the device
  - Describes
    - Pins
    - Use cases
    - Basic layouts
    - Some registers
  - This may be it depending on the device (especially simpler ones)
- Technical Reference Manuals (TRMs)
  - A totalized documentation of the device
  - More in depth than Datasheets
  - Primarily found for MCUs
- Hardware Reference Manual
  - Gives in depth application examples
  - Outlines specific electrical expectations for the device

Reading Datasheets

The RP2350 datasheet is 1371 pages. This is far from the largest datasheet you will come across. But be not afraid! If you know what to look for and how to look for it then you will not need 30% of this!

The table of contents is the best starting point.



If the datasheet is shorter, you can skip this step and go straight to using “ctrl+f”!

1. Introduction 13
1.1. The chip 13
1.2. Pinout reference 15
1.2.1. Pin locations 15
1.2.2. Pin descriptions 16
1.2.3. GPIO functions (Bank 0) 17
1.2.4. GPIO functions (Bank 1) 20
1.3. Why is the chip called RP2350? 21
1.4. Version History 21
2. System bus 22
2.1. Bus fabric 22
2.1.1. Bus priority 23
2.1.2. Bus security filtering 23
2.1.3. Atomic register access 23
2.1.4. APB bridge 24
2.1.5. Narrow IO register writes 24
2.1.6. Global Exclusive Monitor 25
2.1.7. Bus performance counters 27
2.2. Address map 27
2.2.1. ROM 27
2.2.2. XIP 28
2.2.3. SRAM 28
2.2.4. APB registers 29
2.2.5. AHB registers 30
2.2.6. Core-local peripherals (SIO) 30
2.2.7. Cortex-M33 private peripherals 31
3. Processor subsystem 32
3.1. SIO 33
3.1.1. Secure and Non-secure SIO 35

[RP2350 Datasheet]

Important Sections

- Register Maps
  - Outlines what each register is for
- Pin Maps
  - Maps pins to uses
- Usage outlines
  - Goes over the expected usage of a certain functionality
- Addresses/Value selects
  - Usually scattered throughout the sheet near their relevant sections

C++ Multi-file Structure

Header Files (*.h / *.hpp)

Header files are used lay out the external interface to interact with a program's implementation.

- Lay out function definitions
- Data types
- Certain variable definitions
- Descriptive comments

While there are many commenting formats, Doxygen style comments are generally the most functional for C/C++.

```
/**
 * Check whether there are bytes available to be read
 * @code
 * if(radio.available()){
 *   radio.read(&data,sizeof(data));
 * }
 * @endcode
 *
 * @see available(uint8_t*)
 *
 * @return True if there is a payload available, false if none is
 *
 * @warning This function relies on the information about the pipe number
 * that received the next available payload. According to the datasheet,
 * the data about the pipe number that received the next available payload
 * is "unreliable" during a FALLING transition on the IRQ pin. This means
 * you should call clearStatusFlags() before calling this function
 * during an ISR (Interrupt Service Routine). For example:
 * @code
 * void isrCallbackFunction() {
 *   bool tx_ds, tx_df, rx_dr;
 *   uint8_t flags = radio.clearStatusFlags(); // resets the IRQ pin to HIGH
 *   radio.available();                        // returned data should now be reliable
 * }
 *
 * void setup() {
 *   pinMode(IRQ_PIN, INPUT);
 *   attachInterrupt(digitalPinToInterrupt(IRQ_PIN), isrCallbackFunction, FALLING);
 * }
 * @endcode
 */
bool available(void);
```

Good Example: https://github.com/nRF24/RF24/blob/master/RF24.h

Libraries

When you include a library in your code, functionally, the code of that library* is copied into your file at compile time.

```
/*
Various data types and helper functions used for radio communication
*/

#pragma once

#include <Arduino.h>
#include <ArduinoEigen.h>
using namespace Eigen;

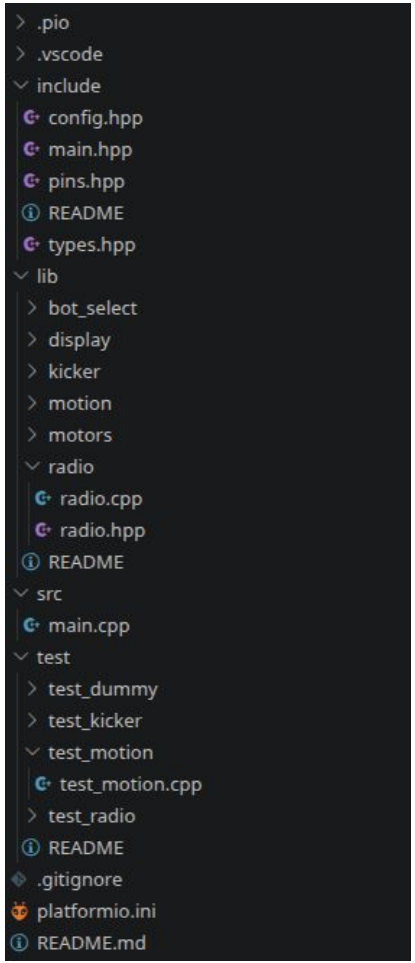
#include "types.hpp"
#include "kicker.hpp"
```

```
#ifndef RADIO_DRIVER_
#define RADIO_DRIVER_
// Library
#endif
```

Important library aspects:

- #pragma once
  - Ensures the library is only loaded once, even if a library also tries to load the library.
  - See below for C implementation
- using <opt> <name>
  - By default C++ will only pull used functions/variables in from library includes
  - “using” lets you specify source
  - “namespace” includes everything from that namespace (can be whole library).

PlatformIO File Layout



- include
  - Header files visible to files in “src”
  - Visibility can be set with build flags
    - “-I include” sets visible everywhere
- lib
  - Library folders
  - Must be nested format
  - good practice to have header and body separate
- src
  - Main program files
- test
  - Tests that can be run with the built in test function
  - Separate tests into directories

Each folder contains a README by default that goes into more detail.

CMakeList

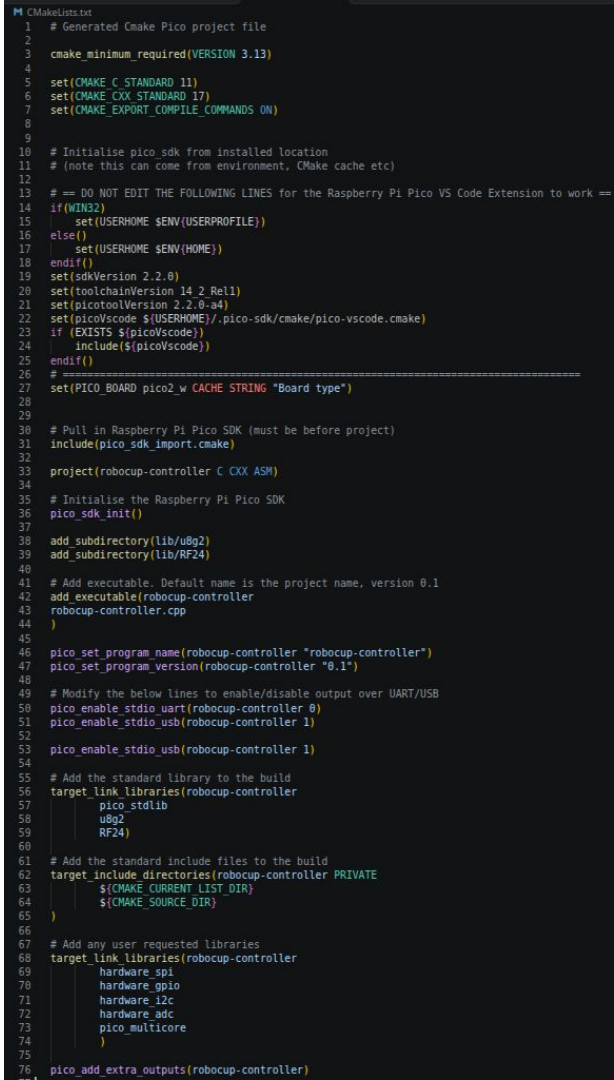
PlatformIO is a wonderful software to use for a lot of microcontroller. However, not all are perfectly supported. Thus, you may also have to work with other code compiler options like Pico SDK, Espressif SDK, etc.

These options do not always have automatic file linking and library importing. What PIO is doing behind the scenes to accomplish this is automatically editing th CMakeList file.

The CMakeList file tells the C compiler where file are and how to link them together.

The actual layout of the CMakeList varies by usage so be sure to research your particular use.

*This can be a decent use case for AI with good direction



DOCUMENT EVERYTHING!!!

But why?

Never, ever assume you will be the last person to edit or use a piece of code. Even if you are, you will likely forget something.

```
45  void Display::do_thing() {
46      uint8_t *b = _u8g2.getBufferPtr();
47      for (uint8_t p = 0; p < 8; p++) {
48          Wire.beginTransmission(0x3C);
49          Wire.write(0x00);
50          Wire.write(0xB0 | p);
51          Wire.write(0x00);
52          Wire.write(0x10);
53          Wire.endTransmission();
54          Wire.beginTransmission(0x3C);
55          Wire.write(0x40);
56          Wire.write(&b[p * 128], 128);
57          Wire.endTransmission();
58      }
59  }
```

```
45  void Display::send_buffer() {
46      uint8_t *buf = _u8g2.getBufferPtr();
47
48      for (uint8_t page = 0; page < 8; page++) {
49          // Tell screen where to write
50          Wire.beginTransmission(0x3C); // Address
51          Wire.write(0x00); // Following bytes are commands
52          Wire.write(0xB0 | page); // Set display page
53          Wire.write(0x00); // Column address lower nibble = 0
54          Wire.write(0x10); // Column address upper nibble = 0
55          Wire.endTransmission();
56
57          // Send actual data
58          Wire.beginTransmission(0x3C);
59          Wire.write(0x40); // Next bytes are display data
60          Wire.write(&buf[page * 128], 128);
61          Wire.endTransmission();
62      }
63  }
```

Code Commenting Basics

Header Comments

```
/**
 * The command is sent from the controller to robot
 *
 * The Packed Format of this message is as follows:
 * +---------+---------+---------+---------+---------+---------+---------+---------+
 * |    7    |    6    |    5    |    4    |    3    |    2    |    1    |    0    |
 * +---------+---------+---------+---------+---------+---------+---------+---------+
 * | unused                                | robot_id                              |
 * +---------+---------+---------+---------+---------+---------+---------+---------+
 * | direction                                                                       |
 * +---------+---------+---------+---------+---------+---------+---------+---------+
 * | speed (lsb)                                                                     |
 * +---------+---------+---------+---------+---------+---------+---------+---------+
 * | speed (msb)                                                                     |
 * +---------+---------+---------+---------+---------+---------+---------+---------+

 * Size = 32 Bits = 4 Bytes
 */
struct command_t {
    // Target robot id
    RobotID id;
    // (dir / 255) * 360 degrees rotation from heading
    uint8_t dir;
    // Forward speed (m/s)
    uint16_t speed;
};

/**
 * Converts a raw radio buffer into a valid robot command
 *
 * @param cmd Input radio buffer
 *
 * @return Command based on buffer input
 *
 * @note Strictly 4 byte input
 *
 * @warning Does not check validity of input
 */
command_t unpack(uint8_t (&cmd)[4]);
```

Body Comments

```
command_t unpack(uint8_t (&cmd)[4]) {
    command_t new_command;
    // Unsafe over allowed ID numbers
    new_command.id = static_cast<RobotID>(cmd[0] & 0xF);
    new_command.dir = cmd[1];
    // cmd[2] lsb | cmd[3] msb
    new_command.speed = ((uint16_t) cmd[3] << 4) | (uint16_t) cmd[2];
    return new_command;
}


// New command to be acted upon next cycle
command_t command;
```

Header comments exhaustively describe the use cases, sizes, units, etc of each aspect.

Body comments are for notes on specific implementations.

Extensive Documentation

Just comments in the code alone go a long way, but to really ensure future usability of what you are making, an external summarized data source is better.

There are two primary ways of doing this:

- README's
  - Automatically format for Github readability
  - Good place to put installation instructions, use cases, and high level information.
  - Uses Markdown formatting
  - Not great for extensive info
- Structured Documents (ie Google Docs)
  - Fully up to you on styling
  - Good for in-depth documentation
  - You have to store and share it

RoboCup Example I

Markdown README's used for installation and usage

README.md > abc # Robocup CPPware

```
# Robocup CPPware

## Description

The 2026 competition brought to light the current short comings of rust in the form of
lacking infrastructure and lengthy debug process. To remedy this a full rewrite of the
firmware stack into C++ was proposed and carried out.

This brings us to the current 3rd rewrite of the Robojackets Robocup firmware.

## Setup

To start clone to repo locally:

```sh
git clone https://github.com/RoboJackets/robocup-cppware
```
Note for Windows you will need some form of Git access such as the [Git Bash]
(https://git-scm.com/install/windows) (easier) or [WSL](https://learn.microsoft.com/
en-us/windows/wsl/install). Or just switch to Linux you will thank yourself later.

There are currently three different frameworks being used to program the robot each
with different processes (This will hopefully be reduced in the future).

Control

The main controller, currently a Teensy 4.1, utilizes the Arduino framework using
Platformio.

The easiest method is to use it with its VSCode extension [here](https://marketplace.
visualstudio.com/items?itemName=platformio.platformio-ide), however versions for other
editors include purely CLI is available on their website [here](https://platformio.org/
platformio-ide).

Kicker

The kicker board, currently a RP2040 (Pico), is programmed using the dedicated
Raspberry PI Pico SDK.

Once again the easiest method is to use the VSCode extension [here](https://marketplace.
visualstudio.com/items?itemName=raspberry-pi.raspberry-pi-pico), however other versions
are available [here](https://www.raspberrypi.com/documentation/microcontrollers/c_sdk.
html).

Motorboard

Currently the motors have not been ported to C++ and still use the rust version, this
should be prioritized to be updated as the installation process is a pain.

Please visit the rust repo for further instructions [here](https://github.com/
RoboJackets/robocup-rustware).

Usage
```

Robocup CPPware

Description

The 2026 competition brought to light the current short comings of rust in the form of lacking infrastructure and lengthy debug process. To remedy this a full rewrite of the firmware stack into C++ was proposed and carried out.

This brings us to the current 3rd rewrite of the Robojackets Robocup firmware.

Setup

To start clone to repo locally:

```
git clone https://github.com/RoboJackets/robocup-cppware
```

Note for Windows you will need some form of Git access such as the Git Bash (easier) or WSL. Or just switch to Linux you will thank yourself later.

There are currently three different frameworks being used to program the robot each with different processes (This will hopefully be reduced in the future).

Control

The main controller, currently a Teensy 4.1, utilizes the Arduino framework using Platformio.

The easiest method is to use it with its VSCode extension here, however versions for other editors include purely CLI is available on their website here.

Kicker

The kicker board, currently a RP2040 (Pico), is programmed using the dedicated Raspberry PI Pico SDK.

Once again the easiest method is to use the VSCode extension here, however other versions are available here.

Motorboard

Currently the motors have not been ported to C++ and still use the rust version, this should be prioritized to be updated as the installation process is a pain.

Please visit the rust repo for further instructions here.

Usage

Control

Ensure you have the entire control folder open in VSCode and select the Platformio icon. Here you will find most of the

RoboCup Example II

Hardware Reference Manuals are used to document PCB structures for easier firmware development, as well as firmware implementation clarification.

Project Title

Hardware Reference Manual

Revision ####
Date
Editor Names

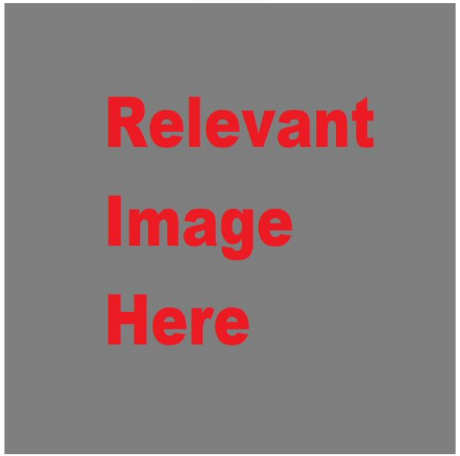
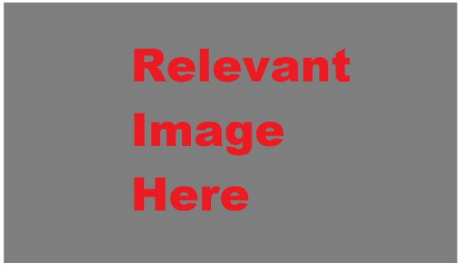


Table of Contents

Table of Contents 1
Board Overview 2
Block Diagram 2
Design Goals 2
Relevant Components 2
Extra Sections 3
Further sections that are needed for clarification or documentation 3
Engineering Change Orders 4
Revision XXX 4
Appendix A: Schematics 5
Appendix B: Manufactured Board Status 6
Appendix C-Onwards: Extra Info 7

Board Overview

Block Diagram



Design Goals

- Relevant
- Goals
- Go
- Here
- Try and outline the purpose of the board and how this will be accomplished in sections.

Relevant Components

- Component
  - Include components which are needed to be known about for others
  - Also include datasheet links

Robocup Example III

Extra Sections

Further sections that are needed for clarification or documentation

Examples:

- Pinouts
- High level section descriptions
- Design recommendations for firmware/software
- Expected use cases

Try to be descriptive but not excessive.
A good example (and what this is based on in general) can be found here.

Engineering Change Orders

Any modifications made post-manufacture are documented here.

Revision XXX

Include detailed description of any and all changes per each revision.

Appendix A: Schematics

Links to schematics for the board(s).

Robocup Example IV

Appendix B: Manufactured Board Status

Keep track of the status of all the boards here, a spreadsheet is recommended.

Appendix C-Onwards: Extra Info

Include any further info needed that is either not needed for general design or is included after the fact in further appendices.

LAB TIME!!!!!

Today's Lab(s)

- IMU Driver
  - Complete the frameworked files to create a functional driver for the IMU
  - Use the IMU's datasheet to accomplish this
  - You will modify /lib/imu/imu(.cpp/.hpp)